



CICLO 1 - 2026

ANÁLISIS Y DISEÑO DE SOFTWARE

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura

Docente: Licdo. Edwin González



Avisos y Anuncios - Semana 1

- Actividades a Desarrollar
 - Participación en el Foro
 - Realizar estudio de contenidos de esta semana
 - Descargar material didáctico de Plataforma



Fundamentos del Análisis y Diseño de Software

Unidad 1



Agenda

1. Fundamentos del Análisis y Diseño de Software

1.1 Ingeniería de Software

1.1.1 Concepto.

1.1.2 Niveles o capas de la Ingeniería de Software.

1.1.3 Crisis del software: Causas y consecuencias

1.1.4 Fases Generales del proceso de desarrollo de software.

1.4.5 Principios de la Ingeniería de Software (abstracción, modularidad, reutilización y calidad).



1.1 Fundamentos del análisis y diseño de software



Análisis y diseño de software



A diferencia de la programación y el diseño a nivel de código, esta habilidad se centra en el diseño de nivel superior, abordando consideraciones de arquitectura y de todo el sistema..

Ingeniería de Software



La ingeniería de software es una disciplina clave en la era digital, que se enfoca en la aplicación de principios de ingeniería al diseño, desarrollo, mantenimiento y evaluación de software. Implica la utilización de métodos sistemáticos, técnicas y herramientas para crear programas informáticos confiables y eficientes, que satisfagan las necesidades específicas de los usuarios y las demandas del mercado.

Introducción



La Ingeniería de Software abarca un amplio espectro de actividades, que van desde la conceptualización y planificación de software hasta su implementación, pruebas y documentación. La ingeniería de software no solo se centra en la programación y el desarrollo de códigos, sino que también hace hincapié en la gestión de proyectos, la calidad del software y la colaboración efectiva entre equipos multidisciplinares.

1.1.1 Concepto



1.1.1 Concepto

La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación. La ingeniería de software es un enfoque sistemático para la producción de software que toma en cuenta los temas prácticos de costo, fecha y confiabilidad, así como las necesidades de clientes y fabricantes de software.



Como este enfoque sistemático realmente implementado varía de manera drástica dependiendo de la organización que desarrolla el software, el tipo de software y los individuos que intervienen en el proceso de desarrollo, no existen métodos y técnicas universales de ingeniería de software que sean adecuados para todos los sistemas y las compañías. (Sommerville, 2011, P.7)



1.1.2 Niveles o capas de la Ingeniería de Software



1.1.2 Capas de la Ingeniería de Software

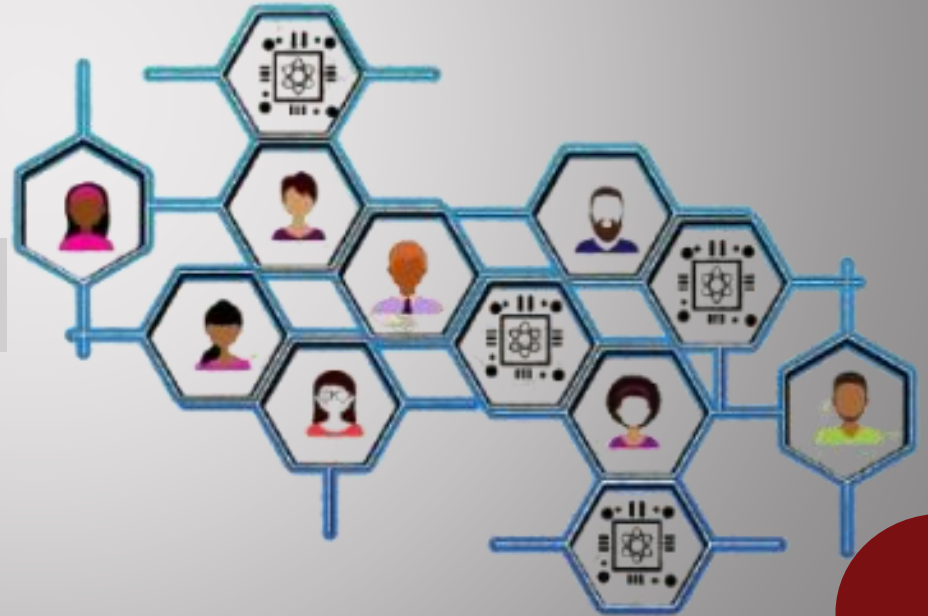
- La ingeniería de software es una tecnología con varias capas. Cualquier enfoque de ingeniería (incluso la de software) debe basarse en un compromiso organizacional **con la calidad**. El fundamento en el que se apoya la ingeniería de software es el compromiso con la calidad.
- El **proceso** de software forma la base para el control de la administración de proyectos de software, y establece el contexto en el que se aplican métodos técnicos, se generan productos, se establecen puntos de referencia, se asegura la calidad y se administra el cambio de manera apropiada.



- Los **métodos** de la ingeniería de software proporcionan la experiencia técnica para elaborar software. Incluyen un conjunto amplio de tareas, como comunicación, análisis de los requerimientos, modelación del diseño, construcción del programa, pruebas y apoyo.
- Las **herramientas** de la ingeniería de software proporcionan un apoyo automatizado o semiautomatizado para el proceso y los métodos. (Pressman, 2010, P.12)



1.1.3 Crisis del software: Causas y consecuencias





La crisis del software

Desde el momento en que se introdujeron computadores con capacidad para soportar aplicaciones de tamaño considerable (años 60), las técnicas de desarrollo para los hasta entonces pequeños sistemas dejaron progresivamente de ser válidas.

Estas técnicas consistían básicamente en codificar y corregir (code-and-fix) que se resume en lo siguiente:

No existe necesariamente una especificación del producto final, en su lugar se tienen algunas anotaciones sobre las características generales del programa. Inmediatamente se empieza la codificación y simultáneamente se va depurando. Cuando el programa cumple con las especificaciones y parece que no tiene errores se entrega.





La crisis del software surge cuando el desarrollo se realiza sin aplicar metodologías de ingeniería de software, especialmente en proyectos con plazos cortos y equipos pequeños. Esto provoca estimaciones incorrectas y procesos poco controlados.

Causas:

- Falta de uso sistemático de metodologías de ingeniería de software.
- Desarrollo improvisado para cumplir tiempos muy cortos.
- Ausencia de planificación y control de calidad.

Consecuencias:

- Aumento del costo por encima de lo previsto.
- Retrasos en el tiempo de entrega.
- Baja calidad del software y presencia de errores.
- Resultados imprevisibles.



1.1.4 Fases del Desarrollo de Software



1.1.4 Fases del Desarrollo de Software

En el contexto de la ingeniería de software, un proceso no es una prescripción rígida de cómo elaborar software; por el contrario, es un enfoque adaptable. Se busca siempre entregar el software en forma oportuna y con calidad suficiente para satisfacer a quienes patrocinaron su creación y a aquellos que lo usarán.

La estructura del proceso establece el fundamento para el proceso completo de la Ingeniería de Software por medio de la identificación de un número pequeño de actividades estructurales que sean aplicables a todos los proyectos de software, sin importar su tamaño o complejidad.

Una estructura de proceso general para la Ingeniería de Software consta de cinco actividades:

- **Comunicación.** Antes de que comience cualquier trabajo técnico, tiene importancia crítica comunicarse. Se busca entender los objetivos de los participantes respecto del proyecto, y reunir los requerimientos que ayuden a definir las características y funciones del software.
- **Planeación.** Define el trabajo de Ingeniería de Software al describir las tareas técnicas por realizar, los riesgos probables, los recursos que se requieren, los productos del trabajo que se obtendrán y una programación de las actividades.

Modelado.

Crea un “bosquejo” del objeto por hacer a fin de entender el panorama general. Un ingeniero de software hace lo mismo al crear modelos a fin de entender mejor los requerimientos del software y el diseño que los satisfará.

Construcción.

Esta actividad combina la generación de código (ya sea manual o automatizada) y las pruebas que se requieren para descubrir errores en éste.

Despliegue.

El software (como entidad completa o como un incremento parcialmente terminado) se entrega al consumidor que lo evalúa y que le da retroalimentación, misma que se basa en dicha evaluación.



1.4.5 Principios de la Ingeniería de Software (abstracción, modularidad, reutilización y calidad).

- Las abstracciones del software representan las diferentes formas en que se entiende un sistema según el rol de quien lo utiliza o desarrolla.

Usuario final :Solo le interesa qué hace el sistema y cómo interactuar con él (no le importa cómo está hecho).

Diseñador: Define cómo va a funcionar internamente usando modelos, algoritmos y estructuras de datos.

Programador: Lo convierte en código usando lenguaje de programación, algoritmos y estructuras de datos.

Mantenedor: Necesita entender la arquitectura, la funcionalidad y la tecnología usada para poder corregir, mejorar o adaptar el sistema.





1.4.5 Principios de la Ingeniería de Software (abstracción, modularidad, reutilización y calidad).

La modularidad es importante en casi todos los productos y procesos de ingeniería. Un sistema complejo puede dividirse en partes llamadas módulos. Un sistema dividido en módulos es modular.





1.4.5 Principios de la Ingeniería de Software (abstracción, modularidad, reutilización y calidad).

Reutilización: Consiste en aprovechar componentes de software ya existentes (código, módulos, librerías, diseños o documentación) en nuevos proyectos o en diferentes partes del mismo sistema.

Beneficios: reduce tiempo de desarrollo, disminuye costos, evita errores y aumenta la productividad.

Calidad: Se refiere a que el software cumpla con los requisitos establecidos y funcione correctamente, siendo confiable, eficiente, seguro, mantenible y fácil de usar. Implica aplicar buenas prácticas, pruebas, estándares y revisiones durante todo el proceso de desarrollo.



Recordatorios

- Investiga más sobre los temas vistos en clase.
- Participa en foros.
- Si tienes dudas, ponte en contacto con tu tutor.



¡Gracias!



Icons made by storyset.com from @flaticon